

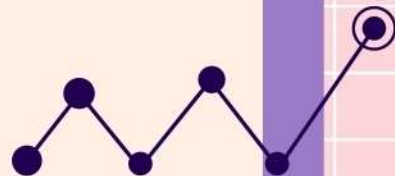
>(□□0△//△/□□)<

Словари, их методы и использование

Программирование в лингвистике

Лекция 4

Словари и основы БД



Словари в Python

```
dct = {1: "a", 2: "b", 3: "c"}
```

* У словаря есть ключи и значения. Здесь 1, 2 и 3 – ключи, а 'a', 'b', и 'c' – значения.

* Ключи работают, как индексы списка. Как элементы множества, ключи могут быть только неизменяемыми, уникальными и неупорядоченными.

Так, в ключах могут быть числа, строки, кортежи, bool и даже None.

Словари в Python – неупорядоченные коллекции пар объектов с доступом по ключу. Их иногда ещё называют ассоциативными массивами или хеш-таблицами.

Значения словаря могут быть любыми. Они могут включать в себя другие словари, списки, множества и т.д. Словари итерируемые и изменяемые.

Как создать словарь?

Способ 1: **явно** в коде.

```
dct = {"sky": "небо", "earth": "земля"}
```

Способ 2: преобразованием в словарь **списка кортежей**.

```
list_of_tuples = [("sky", "небо"), ("earth", "земля")]  
dct = dict(list_of_tuples)  
dct
```

```
{'sky': 'небо', 'earth': 'земля'}
```



Как создать словарь?

Способ 3: преобразованием объектов, возвращаемых функциями `zip(list1, list2)` и `enumerate(list)` (они возвращают списки кортежей)

`zip()`

```
lst1 = ["sky", "earth"]
lst2 = ["небо", "земля"]
dct = dict(zip(lst1, lst2))
dct
```

```
{'sky': 'небо', 'earth': 'земля'}
```

`enumerate()`

```
lst = ["zero", "one", "two"]
dct = dict(enumerate(lst))
dct
```

```
{0: 'zero', 1: 'one', 2: 'two'}
```

Как создать словарь?

Способ 4: задать ключи и их дефолтное значение с помощью `dict.fromkeys([key1, key2, ...], default_value)`

```
dct = dict.fromkeys(['one', 'two', 'three'], 0)
dct
```

```
{'one': 0, 'two': 0, 'three': 0}
```

Как создать словарь?

Способ 5: с помощью **dict comprehension** (генератор словарей).



```
squares = {x: x ** 2 for x in range(2, 6)}  
squares
```

```
{2: 4, 3: 9, 4: 16, 5: 25}
```

Обращение к значению по ключу

Получение значения по ключу – просто и удобно.

Нотация напоминает обращение к элементу списка по индексу.

```
dct = {"sky": "небо", "earth": "земля"}  
print(dct["sky"])  
print(dct["earth"])
```

```
небо  
земля
```

Обращение к значению по ключу

Метод `.get()` позволяет сделать то же самое, только страхует нас в том случае, если **такого ключа нет в словаре**.

```
dct = {"sky": "небо", "earth": "земля"}  
print(dct.get("grass"))  
print(dct["grass"])
```

None

KeyError

```
print(dct.get("sky"))
```

небо

<- Обращение по несуществующему ключу вызывает **KeyError**. А `.get()` возвращает **None**.

Добавление элемента в словарь

Добавить элемент можно просто операцией присваивания – `dict[key] = value`.

```
dct = {"one": 1, "two": 2}
```

```
dct["three"] = 3
```

```
dct
```

Если такого ключа нет – ничего страшного. Однако, любая операция с отсутствующим ключом приведет к **KeyError**.

```
{'one': 1, 'two': 2, 'three': 3}
```

Наличие ключа

Наличие ключа проверяется с помощью **in**:

```
dct = {"one": 1, "two": 2, "three": 3}
print('one' in dct)
print('one' not in dct)
```

True

False

View objects и словари

Итак, словарь состоит из **ключей** и **значений**. В Python существуют «взгляды» на объекты – **view objects**.

```
dct = {"sky": "небо", "earth": "земля"}  
print(dct.keys())  
print(dct.values())  
print(dct.items())
```

```
dict_keys(['sky', 'earth'])  
dict_values(['небо', 'земля'])  
dict_items([('sky', 'небо'), ('earth', 'земля')])
```

Мы можем просмотреть часть словаря – **ключи** или **значения**, а также **пары** (ключ, значение). Мы не можем **изменять** их, но можем **обратиться** к ним.

Например, мы можем итерироваться по **ключам**, **значениям** или **парам**.

Итерируемся по словарю

Только **по ключам**:

```
dct = {"one": 1, "two": 2, "three": 3}
for key in dct:
    print(f'Ключ: {key}, значение: {dct[key]}')
```

Ключ: one, значение: 1

Ключ: two, значение: 2

Ключ: three, значение: 3

Итерируемся по словарю

То же самое:

```
dct = {"one": 1, "two": 2, "three": 3}
for key in dct.keys():
    print(f'Ключ: {key}, значение: {dct[key]}')
```

Ключ: one, значение: 1

Ключ: two, значение: 2

Ключ: three, значение: 3

Итерируемся по словарю

Только по значениям:

```
dct = {"one": 1, "two": 2, "three": 3}
for value in dct.values():
    print(f'Значение: {value}')
```

Значение: 1

Значение: 2

Значение: 3



Итерируемся по словарю

По парам – кортежам (ключ, значение):

```
dct = {"one": 1, "two": 2, "three": 3}
for key, value in dct.items():
    print(f'Ключ: {key}, значение: {value}')
```

Ключ: one, значение: 1

Ключ: two, значение: 2

Ключ: three, значение: 3



Пример

```
monthslist = ['January', 'February', 'March', 'April',  
              'May', 'June', 'July', 'August', 'September',  
              'October', 'November', 'December']  
months = dict(enumerate(monthslist))  
print('Что-то не очень, да?', months)
```

Что-то не очень, да? {0: 'January', 1: 'February', 2: 'March',

Здесь **enumerate()** как всегда подставит каждому элементу значение из **range(0, len(list) + 1)**. Нам это не подходит.

Пример

```
months = {x + 1: y for x, y in enumerate(monthslist)}  
print('Вот так уже лучше...')  
for key, value in months.items():  
    print(f'{value} is number {key}')
```

Вот так уже лучше...
January is number 1
February is number 2
March is number 3
April is number 4

Здесь мы **искусственно**
увеличиваем каждый номер
enumerate() на 1.

Пример

```
months = dict(enumerate(monthslist, 1))  
print('Вот теперь совсем хорошо')  
for key, value in months.items():  
    print(f'{value} is number {key}')
```

```
Вот теперь совсем хорошо  
January is number 1  
February is number 2  
March is number 3  
April is number 4  
May is number 5  
June is number 6  
July is number 7  
August is number 8
```

Здесь мы передаем в `enumerate` аргумент – число, с которого начинается нумерация



Длина словаря

К словарю применима функция `len()`.

```
dct = {"one": 1, "two": 2, "three": 3}  
len(dct)
```

3

Удаление элемента

Способы 1 и 2. С помощью `.pop()` и `.popitem()`

```
dct = {"one": 1, "two": 2, "three": 3}
value = dct.pop("one")
print(value)
```

1

```
key_value = dct.popitem()
key_value
('three', 3)
```

`.pop()` возвращает значение удаленного ключа.

`.popitem()` удаляет последний добавленный элемент и возвращает пару (ключ, значение)



Удаление элемента

Способ 3. С помощью **del**.

```
dct = {"one": 1, "two": 2, "three": 3}
```

```
del dct["one"]
```

```
dct
```

```
{'two': 2, 'three': 3}
```



Соединение словарей

```
dct1 = {"a": 1, "b": 2}
dct2 = {"b": 3, "c": 4}
dct1.update(dct2)
dct1
{'a': 1, 'b': 3, 'c': 4}
```

In-place метод **.update** соединяет два словаря.

Если во втором словаре содержатся ключи, которые есть в первом, то значение ключа в первом словаре **перезапишется** значением второго.

Распаковка и упаковка

****dct** – распаковка словаря

- Превращает словарь в набор **отдельных элементов**.
- Эти элементы потом можно передать в другой словарь, объединив два словаря.
- Так же мы можем передать в функцию **набор именованных аргументов**.

```
dct1 = {'a': 1, 'b': 2, 'c': 3}
```

```
dct2 = {'d': 4, 'e': 5}
```

```
dct3 = {**dct1, **dct2}
```

```
dct3
```

```
{'a': 1, 'b': 2, 'c': 3, 'd': 4, 'e': 5}
```

Распаковка применима не только к словарю, но и к **любому итерируемому объекту** — множеству, списку, кортежу, генератору...

Распаковка и упаковка

*tpl – распаковка кортежа

- Неявно используется при реализации множественного присваивания.
- Тоже удобна для объединения кортежей.
- Так можно передать в функцию **набор позиционных аргументов**.

```
tpl1 = ('a', 'b', 'c')
tpl2 = ('d', 'e')
tpl3 = (*tpl1, *tpl2)
tpl3
('a', 'b', 'c', 'd', 'e')
```

Если распаковать словарь как ***dct**, получим набор ключей, а если как ***dct.values()**, получим набор значений. Чтобы получить **пары** – только ****dct**.

Распаковка и упаковка

```
var, *lst = dct.values()
```

- Тот же синтаксис в другом контексте интерпретируется как **запаковка**.
- Позволяет **значение только первого ключа** из словаря записать в **переменную**, а из остальных создать **другой итерируемый объект**.

```
dct = {'a': 1, 'b': 2, 'c': 3}
```

```
x, *lst = dct.values()
```

```
lst
```

```
[2, 3]
```

***args** часто используется при создании **гибких функций**, способных принимать наборы позиционных аргументов разной длины.

Словари и БД

- Словари тесно связаны с таким разделом программирования как **работа с базами данных**.
- **База данных (БД)** — место, где хранятся **все данные** (на сервере, в файле, в облаке); представляет собой множество связанных между собой **таблиц**.
- **Словарь** — временное представление **одной записи** из БД для локального хранения в памяти и обработки программой.
- **Список словарей** — локальная **копия таблицы** (или ее части) из БД.
- Упрощенно это можно представлять так, что база данных — это огромный шкаф с папками, а словарь — это одна папка, которую вы достали, чтобы поработать.
- Данные извлекаются из базы посредством **SQL-запросов** и записываются в словарь или список словарей.

Словари и БД

БАЗА ДАННЫХ (таблица `students`)

id	name	city	scores
1	Анна	Москва	85
2	Борис	Санкт-Петербург	92
3	Вера	Казань	78

▼ `SELECT * FROM students WHERE id=2`

```
student = {  
  "id": 2,  
  "name": "Борис",  
  "city": "СПб"  
}
```

- * Если программа получает **одну запись**, то она создает **словарь**.
- * Если она получает **несколько записей**, то она создает **список словарей**.
- * Так же можно извлечь и все данные таблицы.

Словари и БД

Результат SQL-запроса: SELECT * FROM students

```
students = [  
    {"id": 1, "name": "Анна", "city": "Москва", "score": 85},  
    {"id": 2, "name": "Борис", "city": "СПб", "score": 92},  
    {"id": 3, "name": "Вера", "city": "Казань", "score": 78},  
]
```

- Мы можем фильтровать и анализировать данные прямо в Python, не делая новый запрос к БД.

Словари и БД

```
high_score = [s for s in students if s["score"] > 80]
print("\nСтуденты с баллом > 80:")
for s in high_score:
    print(f" {s['name']} — {s['score']}")
max_score = max(s["score"] for s in students)
print(f"Максимальный балл: {max_score}")
```

Студенты с баллом > 80:

Анна — 85

Борис — 92

Максимальный балл: 92

Можем найти:

- студентов с баллом выше 80
- максимальный балл

Можем посчитать:

- средний балл по группе
- количество студентов из Москвы

**Спасибо за
внимание!**